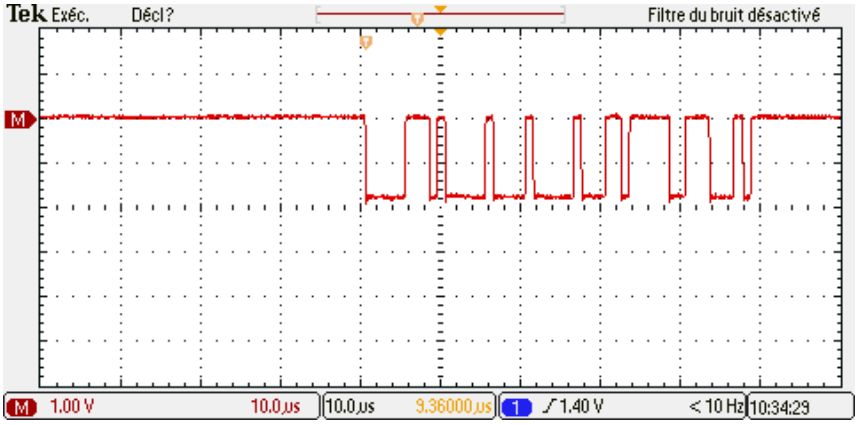
HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

Hes·so
Haute Ecole Spécialisée de Suisse occidentale
Fachhochschule Westschweiz
University of Applied Sciences and Arts Western Switzerland

Laboratoire 4 :

Communication sur le bus CAN

Département : Technologies industrielles (TIN)
Unité d'enseignement : Bus de terrain



Auteurs : Tom Berthoud & Luc Marchand
Professeur : François Birling
Assistant : Andrea Bernasconi & Gaëtan Worch
Classe : A
Date : 24 avril 2026

Table des matières

1 Introduction **3**

2 Matériel et protocole applicatif **3**
2.1 Matériel 3
2.2 Format des messages CAN 3
2.3 Configuration par DIP-switch 4

3 Programmation **5**
3.1 Architecture du programme 5
3.2 Masques et filtres de réception 5
3.3 Gestion des LEDs d'état 5

4 Mesures sur bus libre à 1 Mbps **6**
4.1 Analyse d'une trame CAN 6
4.2 Comportement en cas de problème de transmission 7
4.3 Latence et gigue à vide 7
4.4 Transmission des valeurs analogiques (trafic spontané) 9
4.5 Transmission d'un signal sinusoïdal 9

5 Mesures sur bus chargé à 200 kbps **12**
5.1 Latence et gigue sous charge 12
5.2 Transmission d'un signal sinusoïdal sur bus chargé 14

6 Conclusion **16**

7 Signatures **17**

A Listing du programme **18**

1 Introduction

Ce laboratoire a pour objectif la mise en œuvre pratique du bus de terrain CAN entre deux cartes Arduino Mega 2560 équipées d'un Shield CAN (MCP2515 + MCP2551) et d'une carte d'entrées/sorties HEIG-VD. Le but est d'écrire un protocole applicatif permettant de recopier sur chaque carte les entrées (tout-ou-rien et analogiques) lues par la carte jumelle.

Le travail se décompose en trois parties :

— **Programmation** du contrôleur CAN MCP2515 à l'aide de la bibliothèque `BusTerUtils` : configuration des masques/filtres de réception, gestion des messages analogiques et tout-ou-rien, lecture des DIP-switches pour l'adressage.

— **Mesures sur un bus libre** à 1 Mbps : analyse d'une trame, comportement lorsque le câble est débranché, mesure de la latence et de la gigue entre deux cartes, puis transmission d'un signal analogique et sinusoïdal.

— **Mesures sur un bus chargé** à 200 kbps avec deux paires de cartes partageant le même bus : étude de l'impact du trafic sur la latence, la gigue et la qualité de reproduction d'un signal sinusoïdal.

Les mesures sont effectuées à l'oscilloscope Tektronix et les oscillogrammes sont intégrés au rapport.

2 Matériel et protocole applicatif

2.1 Matériel

Chaque station est constituée des éléments suivants :

— une carte **Arduino Mega 2560** (microcontrôleur ATmega2560) ;

— un **Shield CAN** construit autour des circuits `MCP2515` (contrôleur CAN autonome, interface SPI) et `MCP2551` (émetteur/récepteur CAN) ;

— une **carte d'E/S HEIG-VD** offrant 8 entrées tout-ou-rien 24 V, 8 sorties PNP 12 V, 4 entrées analogiques 0–5 V, 4 sorties analogiques 0–5 V ainsi qu'un DIP-switch à 8 interrupteurs et une LED bicolore d'état.

Les cartes sont reliées entre elles par un câble Sub-D 9 pôles transportant les signaux `CANH`, `CANL` et `GND`, conformément au brochage CANopen. Les résistances de terminaison de 120 Ω intégrées au Shield ayant été désactivées, des résistances externes ont été placées aux deux extrémités du bus.

2.2 Format des messages CAN

Le protocole utilise le format standard **CAN 2.0A**, avec un identifiant sur 11 bits. Cet identifiant est découpé en deux parties :

Bits	Rôle	Valeurs
b10 – b7	Type de message	0000 = analogique, 0001 = tout-ou-rien
b6 – b0	Adresse de la carte (7 bits)	DIP2–DIP7, avec inversion du DIP1

Table 1 – Découpage de l'identifiant CAN sur 11 bits

Deux types de trames sont utilisés :

Type	ID	Contenu des données
0000	`0x000` + adresse (7 bits)	4 valeurs analogiques codées sur 16 bits (8 octets) : AIN0_LOW, AIN0_HIGH, ..., AIN3_LOW, AIN3_HIGH
0001	`0x080` + adresse (7 bits)	1 octet regroupant les 8 entrées tout-ou-rien

Table 2 – Types de messages CAN définis pour l'application

Un message n'est émis que si les valeurs associées ont *changé* depuis le dernier envoi, ce qui limite le trafic sur le bus.

2.3 Configuration par DIP-switch

Les 8 interrupteurs du DIP-switch ont la fonction suivante :

DIP 1	Bit de poids faible de l'adresse pour les messages *reçus*. Son *inverse* est utilisé pour les messages *émis*. Seul ce bit diffère entre deux cartes jumelées.
DIP 2–7	6 bits de poids supérieur de l'adresse (DIP7 = MSB).
DIP 8	Active (1) ou désactive (0) l'émission / la réception des messages analogiques. Les messages tout-ou-rien sont toujours traités.

Table 3 – Affectation des 8 interrupteurs DIP

Exemple : la carte 1 avec DIP = `0x3A` reçoit les messages portant l'adresse `0x3A` et émet avec l'adresse `0x3B`. La carte 2 (DIP = `0x3B`) fait exactement l'inverse, ce qui assure la communication bidirectionnelle.

3 Programmation

3.1 Architecture du programme

Le même programme est chargé dans les deux cartes jumelles ; l'adressage est déterminé au démarrage par la lecture du DIP-switch. Le code s'organise en deux parties :

— `setup()` : initialisation des E/S (`BusTer_InitInputOutput`), lecture du DIP-switch, calcul des adresses de réception et d'émission, initialisation du MCP2515 à la vitesse choisie, programmation des masques et filtres, puis passage en mode normal.

— `loop()` : lecture des 8 entrées tout-ou-rien et émission d'un message de type 1 si leur valeur a changé ; si DIP8 = 1, lecture des 4 entrées analogiques et émission d'un message de type 0 si au moins une valeur a changé ; réception éventuelle dans RXB0 (tout-ou-rien) et dans RXB1 (analogiques), avec recopie sur les sorties. En cas d'erreur bus, allumage de la LED rouge et acquittement.

3.2 Masques et filtres de réception

L'énoncé impose que les messages tout-ou-rien soient reçus dans `RXB0` et les messages analogiques dans `RXB1`. Les masques et filtres du MCP2515 ont été réglés comme suit :

Registre	Masque	Filtre(s)
RXB0 (digital)	`0x7FF` (tous les bits comparés)	`RXF0=RXF1=rxAddress`
RXB1 (analogique)	`0x7FF`	`RXF2..RXF5= 0x080 | rxAddress`

Table 4 – Configuration des masques et filtres

Comme l'indique la note de l'énoncé, *tous* les registres de filtres ont été initialisés, faute de quoi les filtres non programmés accepteraient les messages d'ID 0 par défaut.

3.3 Gestion des LEDs d'état

La LED bicolore de la carte signale l'état de la communication :

— **verte** : contrôleur initialisé, pas d'erreur CAN ;

— **rouge** : erreur bus signalée par le MCP2515. L'erreur est acquittée automatiquement via `BusTer_CanClearError()`.

Le listing complet du programme figure en **Annexe A**.

4 Mesures sur bus libre à 1 Mbps

Les premières mesures sont réalisées avec une seule paire de cartes A1/A2 et une vitesse de bus de 1 Mbps.

4.1 Analyse d'une trame CAN

Pour cette mesure, le DIP8 est mis à 0 (pas de trafic analogique). L'entrée tout-ou-rien n°0 est basculée pour provoquer l'émission d'un unique message. La figure 1 montre la trame CAN capturée à l'oscilloscope sur la ligne `CANH`. On y distingue les différents champs caractéristiques d'une trame CAN 2.0A : **SOF** (1 bit dominant), **champ d'arbitration** (11 bits d'identifiant + RTR), **champ de contrôle** (IDE, r0, DLC), **champ de données** (ici 1 octet), **CRC** (15 bits + délimiteur), **ACK** (slot + délimiteur) et enfin **EOF** (7 bits récessifs).

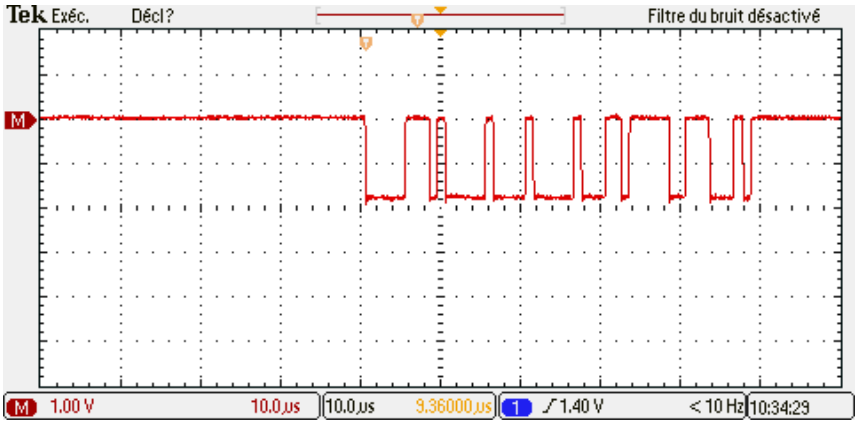


FIGURE 1 – Trame CAN complète capturée sur `CANH` à 1 Mbps (base de temps 10 µs/div).

La figure 2 permet de vérifier la durée d'un bit à l'aide des curseurs de l'oscilloscope : $\Delta x = 1{,}000$ µs entre deux fronts correspondant à un bit, soit une fréquence de bit de $f_{\text{bit}} = 1{,}00$ MHz ou 1 Mbps, conforme à la configuration.

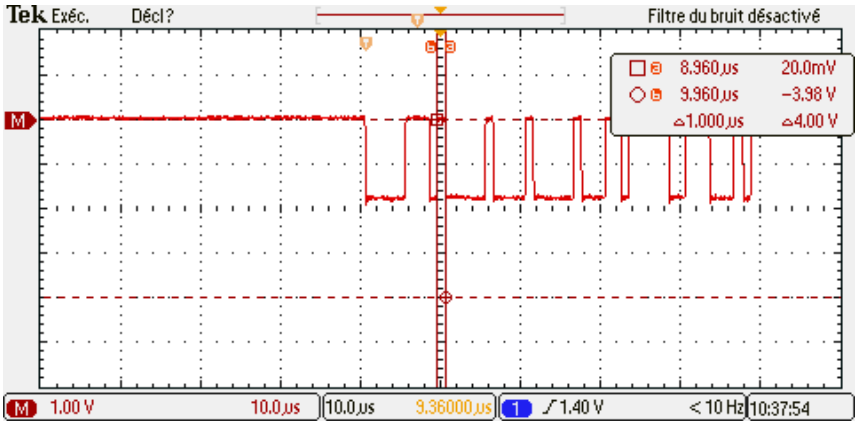


FIGURE 2 – Mesure de la durée d'un bit : $\Delta x = 1{,}00$ µs $\Rightarrow$ 1 Mbps.

4.2 Comportement en cas de problème de transmission

Le câble CAN est débranché, puis l'émission d'un message est provoquée. On observe sur la figure 3 une activité *continue* sur la ligne du côté émetteur : le MCP2515 n'obtient jamais d'ACK du receveur et *retransmet indéfiniment* le même message. La LED rouge s'allume brièvement à chaque erreur de transmission.

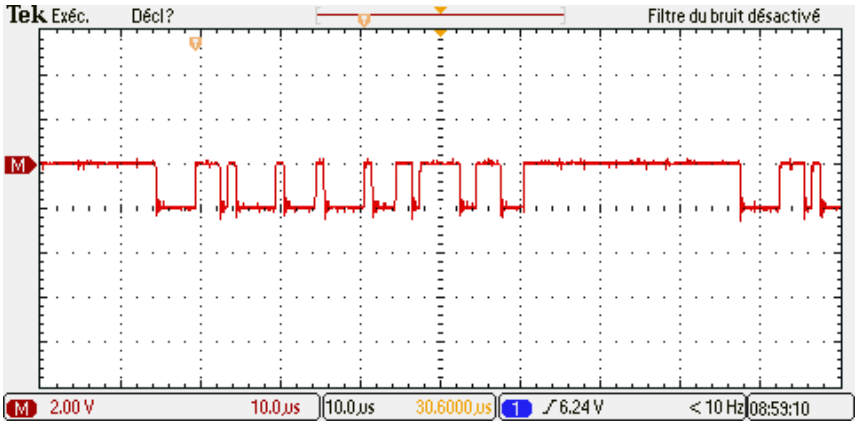


FIGURE 3 – Signal du côté émetteur, câble CAN débranché : retransmission permanente de la trame.

Ce comportement correspond bien à celui attendu du protocole CAN en mode *normal* (non one-shot) : tant que la trame n'a pas été acquittée, le contrôleur réessaie, et l'erreur de bus est levée. C'est pour cette raison que le MCP2515 propose un mode *one-shot* qui limite à une seule tentative pour les protocoles à contrainte temps réel stricte.

4.3 Latence et gigue à vide

Un signal carré à 500 Hz est injecté sur une entrée tout-ou-rien de la carte A1, et le signal récupéré sur la sortie correspondante de la carte A2 est observé simultanément à l'oscilloscope. La latence mesurée est le décalage entre le front d'entrée et le front de sortie.

Grandeur	Valeur mesurée
Latence minimale $t_{\min}$	416 µs
Latence maximale $t_{\max}$	1,05 ms
Gigue $J = t_{\max} - t_{\min}$	$\approx$ 632 µs

TABLE 5 – Latence et gigue à 1 Mbps, bus sans trafic.

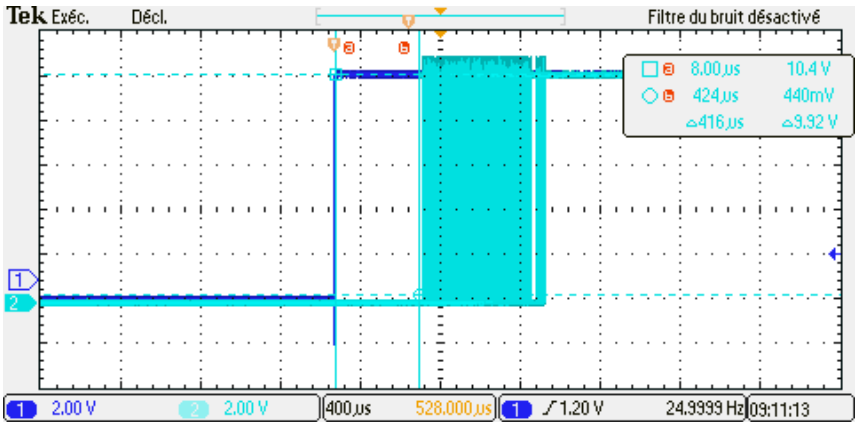


FIGURE 4 – Latence minimale $t_{\min} = 416$ µs (curseurs de l'oscilloscope).

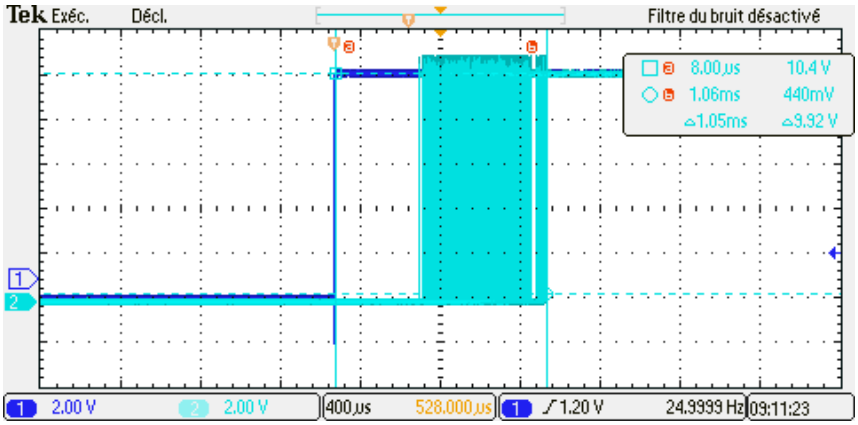


FIGURE 5 – Latence maximale $t_{\max} = 1{,}05$ ms.

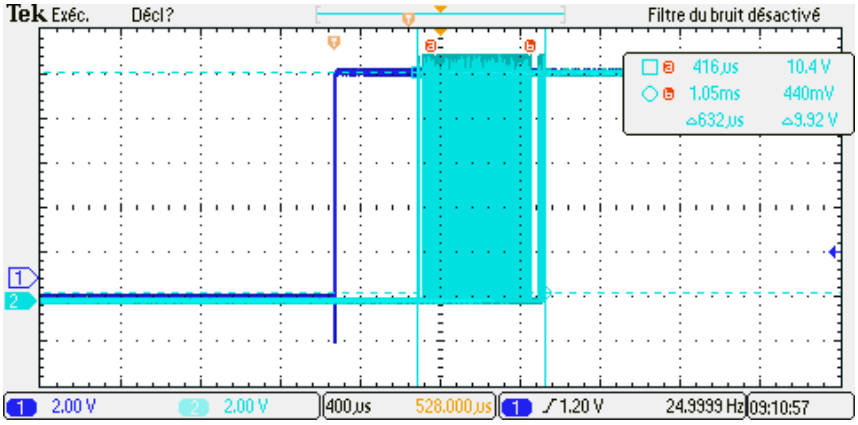


FIGURE 6 – Superposition en persistance infinie mettant en évidence la gigue $\Delta t \approx$ 632 µs.

Interprétation. La latence $t_{\min}$ correspond au temps nécessaire à la détection du front d'entrée, à la construction et à l'envoi du message CAN (8 octets de trame

≈ 100 bits à 1 Mbps, soit ≈ 100 µs), auquel s'ajoute la durée du cycle logiciel `loop()` et le délai de la sortie analogique sur l'autre carte. La gigue, proche de $t_{\min}$, s'explique par le fait que la détection du front peut survenir à n'importe quel instant du cycle `loop()` ; elle est donc bornée approximativement par la durée d'un tour de boucle.

4.4 Transmission des valeurs analogiques (trafic spontané)

Le DIP8 de la carte A1 est mis à 1 pour activer l'émission des messages analogiques, tandis que l'entrée analogique est laissée à un potentiel fixe. La figure 7 montre pourtant un trafic *continu* sur le bus.

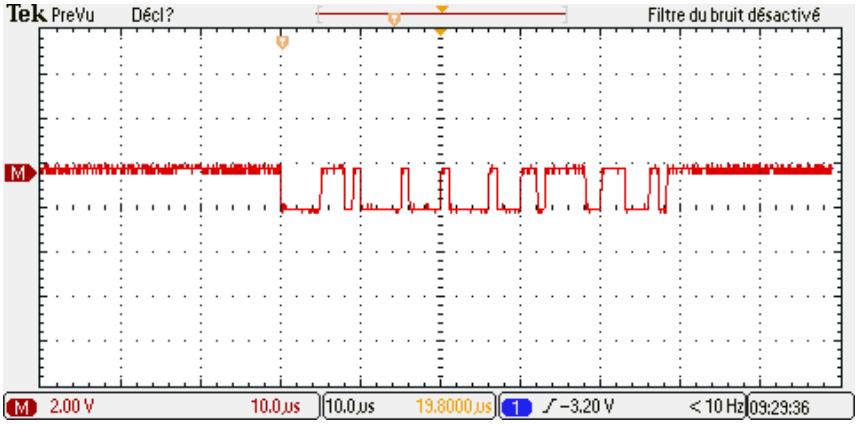


FIGURE 7 – Trafic observé avec l'entrée analogique laissée fixe.

Explication. Bien que l'entrée soit branchée sur un potentiel fixe, le bruit électronique du CAN de l'Arduino Mega fait varier la valeur lue à chaque itération (LSB instable). Comme notre critère d'émission est « envoyer si la valeur a changé depuis le dernier message », un message est généré quasiment à chaque tour de boucle. Pour limiter ce phénomène il faudrait soit filtrer / arrondir la valeur analogique, soit introduire une hystérésis avant de déclencher un envoi.

4.5 Transmission d'un signal sinusoïdal

Un signal sinusoïdal est injecté sur l'entrée analogique AIN0 de la carte A1 et le signal est observé simultanément en entrée (A1) et en sortie analogique (AOUT0 de la carte A2).

Intervalle entre messages. La figure 8 met en évidence, en zoomant sur les marches d'escalier de la sortie, l'intervalle temporel entre deux messages analogiques reçus : $\Delta t \approx 1{,}10$ ms.

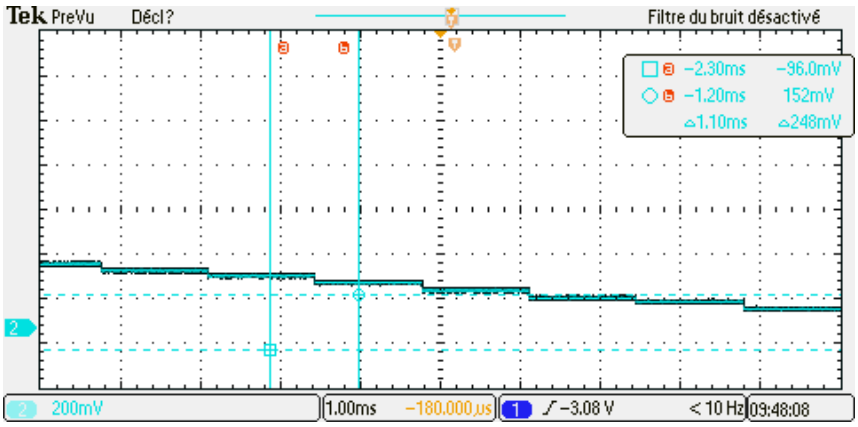


FIGURE 8 – Intervalle entre deux mises à jour successives de la sortie analogique : 1,10 ms.

Qualité à basse fréquence (6 Hz). Pour $f = 6$ Hz, la sortie analogique reproduit fidèlement l'entrée sinusoïdale (figure 9). L'escalier n'est pas visible à cette échelle temporelle, la période étant très grande devant l'intervalle de 1,1 ms entre messages.

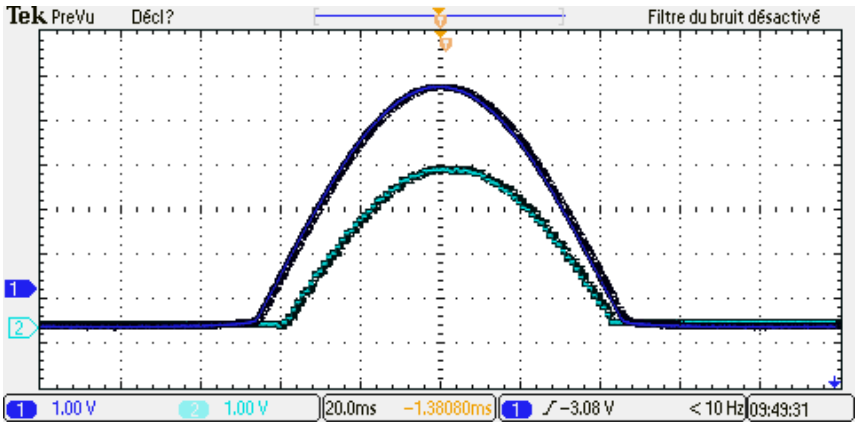


FIGURE 9 – Signaux d'entrée (bleu) et de sortie (turquoise) à $f = 6$ Hz.

Qualité à plus haute fréquence (30 Hz). Pour $f = 30$ Hz (figure 10), la sortie présente de nettes marches d'escalier, un déphasage d'environ un quart de période et une amplitude réduite. On se rapproche de la limite de la fréquence de Shannon : une période de 33 ms ne contient qu'environ 30 échantillons, ce qui dégrade fortement la qualité de reproduction.

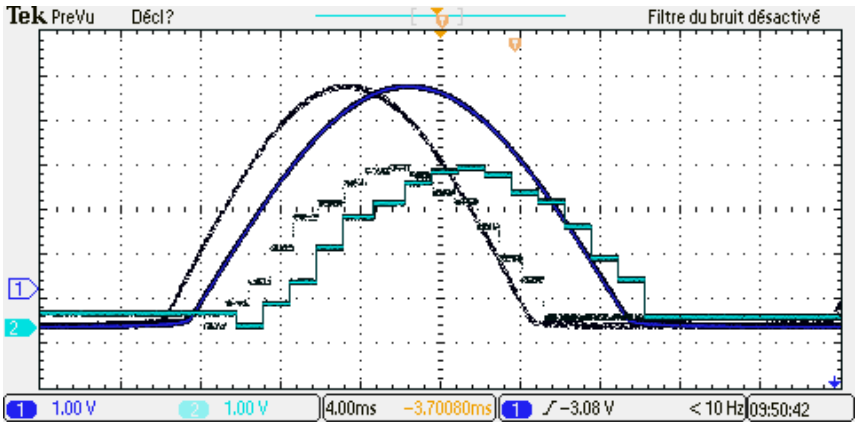


FIGURE 10 – Signaux d'entrée (bleu) et de sortie (turquoise) à $f = 30$ Hz. La sortie est visiblement dégradée et retardée.

Influence de la fréquence. La qualité de reproduction est directement liée au nombre d'échantillons par période du signal sinusoïdal. L'intervalle entre messages étant fixe ($\approx$ 1,1 ms), la dégradation devient perceptible dès que la période du signal se rapproche de cette durée, typiquement au-delà de quelques dizaines de hertz.

5 Mesures sur bus chargé à 200 kbps

Pour cette partie, deux paires de cartes (A1/A2 et B1/B2) partagent le même bus CAN, la vitesse est réduite à 200 kbps et les adresses sont fixées comme suit :

Carte	Adresse (DIP1 → DIP7)
A1	22 (0110 100)
A2	23 (1110 100)
B1	72 (0001 001)
B2	73 (1001 001)

Table 6 – Adresses configurées pour les mesures à 4 cartes.

5.1 Latence et gigue sous charge

Un signal carré à 500 Hz est injecté sur une entrée tout-ou-rien de A1 et la sortie correspondante est observée sur A2. Dans un premier temps, seul le couple A1/A2 est actif, puis le même signal est ajouté sur B1 afin d'introduire du trafic supplémentaire.

Configuration	$t_{\min}$	$t_{\max}$	Gigue J
Couple A1/A2 seul (sans trafic)	604 µs	800 µs	≈ 192 µs
A1/A2 + trafic sur B1/B2	604 µs	1,07 ms	≈ 468 µs

Table 7 – Latence et gigue à 200 kbps avec et sans trafic concurrent.

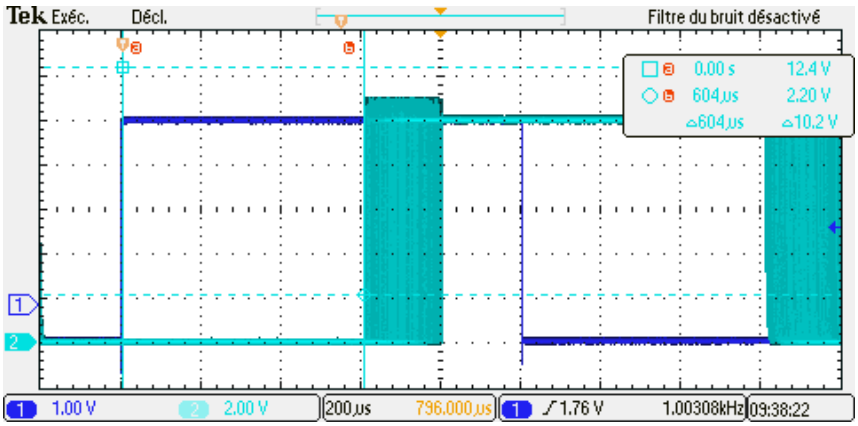


Figure 11 – Latence minimale A1→A2, bus libre : 604 µs.

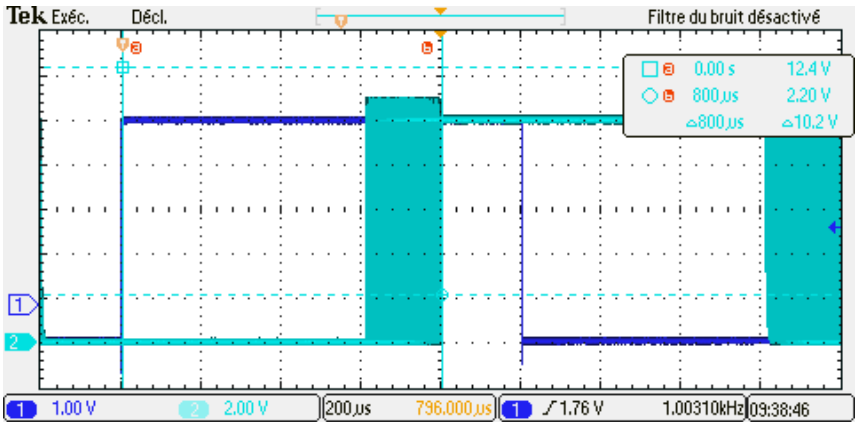


FIGURE 12 – Latence maximale A1→A2, bus libre : 800 µs.

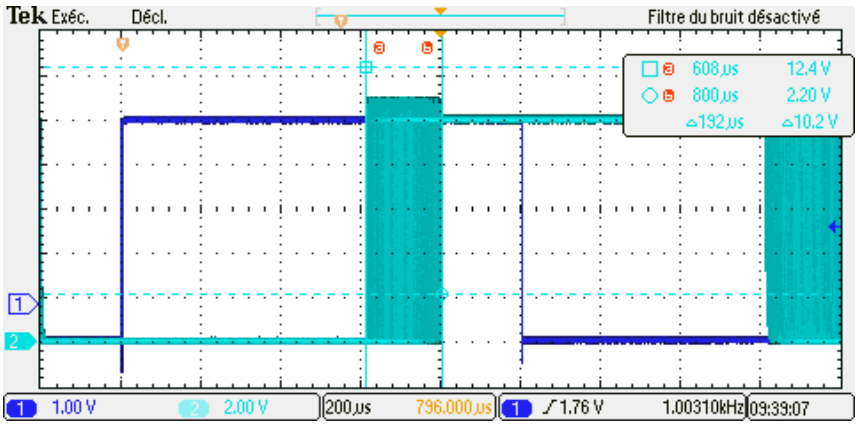


FIGURE 13 – Gigue A1→A2 bus libre : $\Delta t \approx 192$ µs.

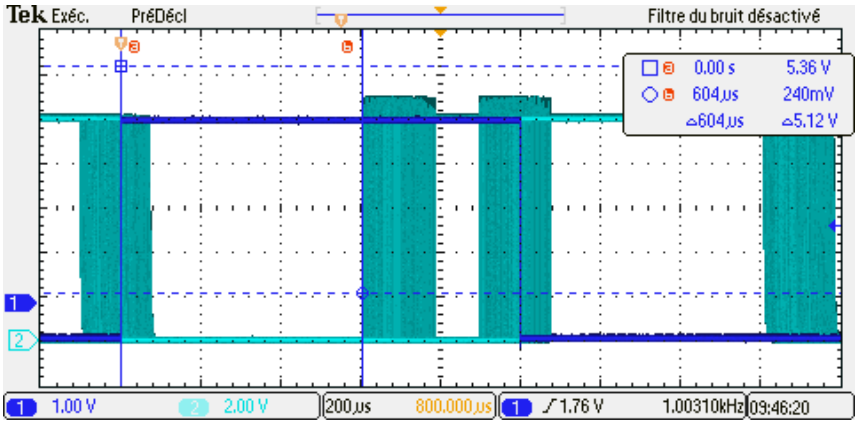


FIGURE 14 – Latence minimale A1→A2 avec trafic B1/B2 : 604 µs.

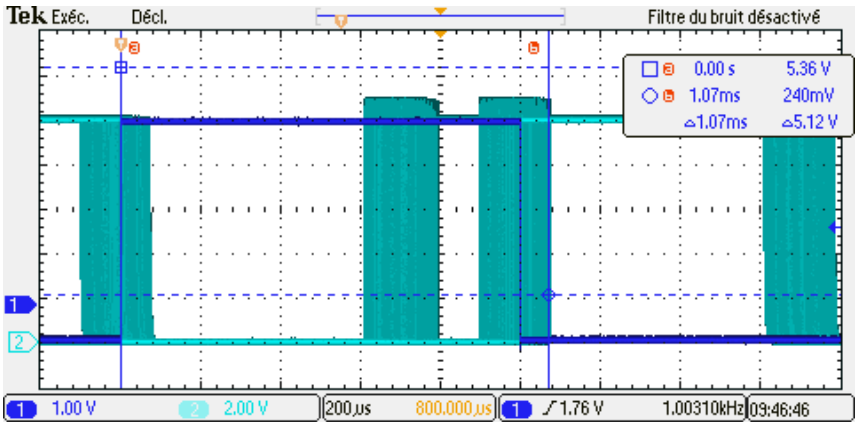


FIGURE 15 – Latence maximale A1→A2 avec trafic B1/B2 : 1,07 ms.

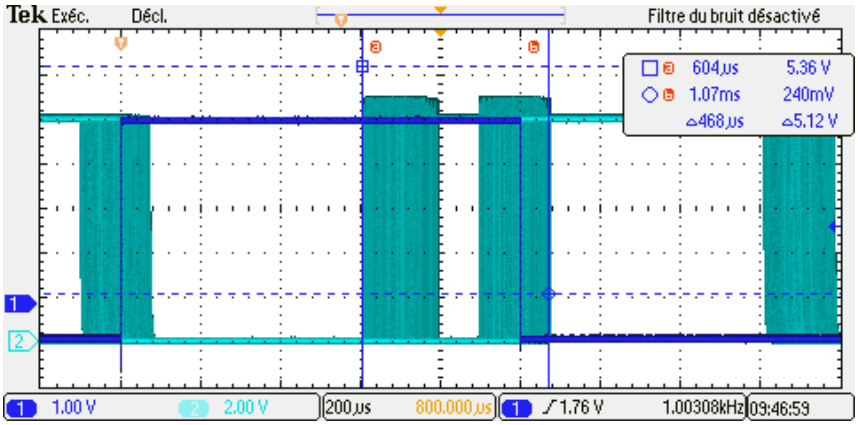


FIGURE 16 – Gigue A1→A2 avec trafic B1/B2 : $\Delta t \approx 468$ µs.

Interprétation.

- La latence minimale reste *constante* (604 µs) : quand le bus est libre au moment de l'émission, la durée d'envoi d'un message est indépendante du trafic potentiel.
- En revanche, la latence maximale augmente (800 µs → 1,07 ms) car le message d'A1 doit parfois *attendre* la fin d'un message concurrent B1→B2 avant d'accéder au bus, conformément au mécanisme d'arbitration non destructive du protocole CAN.
- La gigue augmente donc fortement (≈ 192 µs → ≈ 468 µs), directement dépendante de la durée d'un message concurrent (≈550 µs à 200 kbps).

5.2 Transmission d'un signal sinusoïdal sur bus chargé

Un signal sinusoïdal à 50 Hz est utilisé dans cette partie ; le bus étant à 200 kbps, l'intervalle entre messages analogiques est plus grand qu'à 1 Mbps, ce qui se traduit par des marches d'escalier déjà bien visibles sur la sortie.

Étape 1 — B2 → B1 seul. Seul ce flux analogique circule. Le signal est reproduit avec marches d'escalier mais sans retard ni instabilité apparente (figure 17).

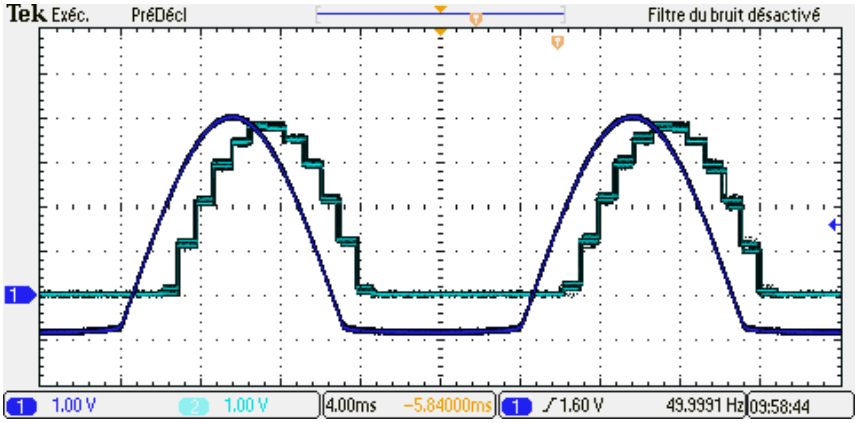


FIGURE 17 – Sinusoïde 50 Hz B2→B1 seul (base 4 ms/div).

Étape 2 — ajout de B1 → B2. On active la sinusoïde dans le sens opposé. Le signal précédent devient *moins stable* (il « bouge » davantage sur l'écran de l'oscilloscope) : chaque message doit désormais partager le bus avec le flux inverse, ce qui introduit une gigue supplémentaire dans l'arrivée des échantillons (figure 18).

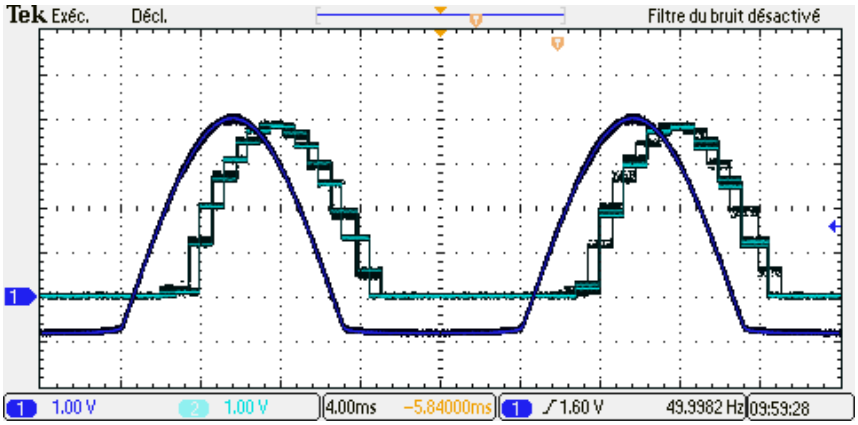


FIGURE 18 – Ajout du flux B1→B2 : la sortie devient moins stable.

Étape 3 — ajout de A2 → A1. Un troisième flux analogique est activé entre les cartes A. La sortie observée se dégrade encore, avec une amplitude réduite et des marches plus grossières (figure 19).

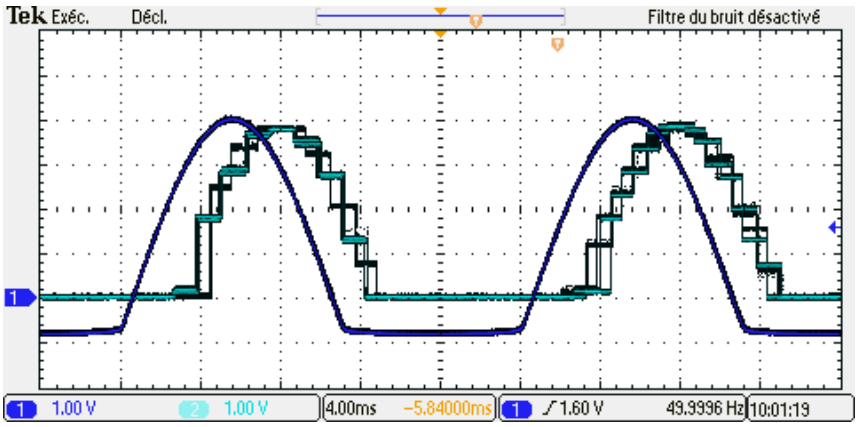


FIGURE 19 – Ajout du flux A2→A1 : dégradation supplémentaire.

Étape 4 — ajout de A1 → A2. Avec les quatre flux analogiques actifs simultanément, le bus est saturé : la sortie observée est *trop en retard*, ne suit quasiment plus la consigne et reste collée à la valeur basse du signal (figure 20). Les messages de mise à jour arrivent si tardivement que la sortie ne « voit » plus les phases hautes de la sinusoïde avant que l'échantillon suivant n'ait déjà changé.

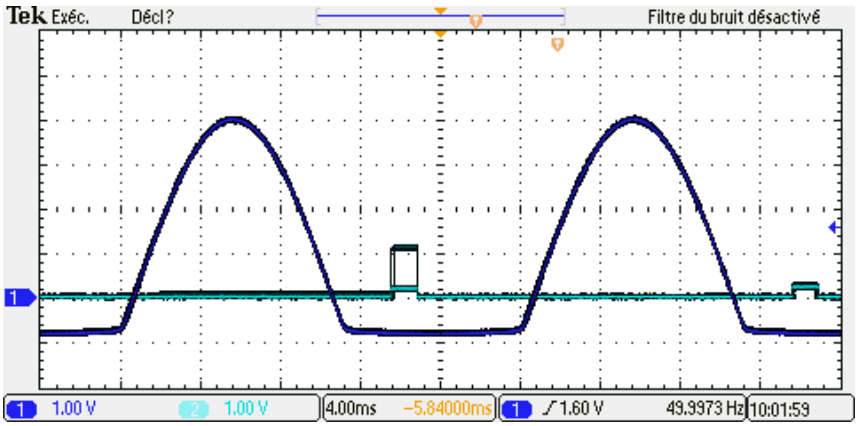


FIGURE 20 – Les 4 flux actifs : le bus est saturé, la sortie est en retard et ne reproduit plus correctement la sinusoïde.

Bilan. Chaque ajout d'un flux analogique (sinusoïde 50 Hz) augmente la charge du bus. Tant que la charge reste modérée, l'arbitration CAN absorbe les collisions et les signaux restent reproduits. Une fois le bus proche de la saturation, l'intervalle entre messages analogiques reçus devient supérieur à la période du signal à transmettre, et la qualité de reproduction s'effondre.

6 Conclusion

Ce laboratoire nous a permis de mettre en œuvre concrètement un protocole applicatif sur bus CAN en programmant un contrôleur MCP2515 depuis une carte Arduino. La configuration des masques et filtres d'acceptation s'est révélée essentielle

pour que chaque type de message soit routé automatiquement dans le bon buffer de réception, sans intervention du processeur dans la boucle principale.

Les mesures à l'oscilloscope ont confirmé plusieurs propriétés fondamentales du protocole CAN :

- la durée d'un bit correspond exactement à la vitesse configurée (1 µs à 1 Mbps) ;
- lorsque la liaison est coupée, l'émetteur retransmet indéfiniment le même message jusqu'à obtention d'un acquittement – ce qui justifie l'existence du mode *one-shot* pour les applications critiques ;
- la latence minimale de propagation est essentiellement déterminée par la durée d'un cycle logiciel et la durée de transmission d'un message, indépendamment de la charge du bus ;
- la latence maximale et la gigue augmentent en revanche avec le trafic concurrent, à cause de l'arbitration non destructive qui force un message à attendre la fin du message en cours avant d'émettre.

L'étude du signal sinusoïdal met en évidence l'importance du choix du *débit* en fonction du contenu applicatif : l'intervalle de 1,1 ms entre messages à 1 Mbps permet une reproduction satisfaisante jusqu'à plusieurs dizaines de hertz, tandis qu'à 200 kbps avec quatre flux concurrents, la sortie analogique devient complètement inutilisable à 50 Hz. Le protocole applicatif « émission uniquement en cas de changement » s'est par ailleurs révélé sensible au bruit du convertisseur analogique, et mériterait l'ajout d'une hystérésis ou d'un filtrage dans une application industrielle.

Ce laboratoire démontre que le dimensionnement d'une liaison CAN ne se résume pas au choix d'une vitesse : le taux de trafic, la longueur des messages, leur priorité relative et le protocole applicatif interagissent tous pour déterminer la qualité effective de la communication.

7 Signatures

Yverdon-les-Bains, 24 avril 2026
Tom Berthoud & Luc Marchand

A Listing du programme

Le code source complet du programme chargé dans les deux cartes jumelles est reproduit ci-dessous.

```
#include <BusTerUtils.h>
#include <SPI.h>
#include <Wire.h>

// Adresses CAN calculées depuis les DIP switches
static uint32_t rxAddress = 0;  // adresse de réception (DIP1-7)
static uint32_t txAddress = 0;  // adresse d'émission (rxAddress XOR 1)
static bool analogMode = false; // DIP8 : gestion de l'analogique

// Dernières valeurs envoyées (pour détecter les changements)
static int    prevAnalog[4]  = {-1, -1, -1, -1};
static int    prevDigital    = -1;

void setup() {
    BusTer_InitInputOutput();

    // Lecture des DIP switches : DIP1=bit0, DIP2=bit1, ..., DIP7=bit6
    rxAddress  = (uint32_t)digitalRead(BUSTER_PIN_DIPSWITCH_1) << 0;
    rxAddress |= (uint32_t)digitalRead(BUSTER_PIN_DIPSWITCH_2) << 1;
    rxAddress |= (uint32_t)digitalRead(BUSTER_PIN_DIPSWITCH_3) << 2;
    rxAddress |= (uint32_t)digitalRead(BUSTER_PIN_DIPSWITCH_4) << 3;
    rxAddress |= (uint32_t)digitalRead(BUSTER_PIN_DIPSWITCH_5) << 4;
    rxAddress |= (uint32_t)digitalRead(BUSTER_PIN_DIPSWITCH_6) << 5;
    rxAddress |= (uint32_t)digitalRead(BUSTER_PIN_DIPSWITCH_7) << 6;
    rxAddress &= 0x7F;

    analogMode = (digitalRead(BUSTER_PIN_DIPSWITCH_8) == HIGH);

    // Adresse d'émission = inversion du bit 0 (DIP1) car nous avons la meme adresse à un bit près
    txAddress = rxAddress ^ 0x01;

    BusTer_CanInit(BUSTER_CAN_1MBPS);

    // --- Masques et filtres ---
    // RXB0 : messages digitaux (type 0, bits[10:7]=0000)
    //        ID attendu = rxAddress (bits[6:0])
    BusTer_CanSetMask(BUSTER_Rxm0, BUSTER_Can2_0A, 0x7FF);
    BusTer_CanSetFilter(BUSTER_Rxf0, BUSTER_Can2_0A, rxAddress);
    BusTer_CanSetFilter(BUSTER_Rxf1, BUSTER_Can2_0A, rxAddress);

    // RXB1 : messages analogiques (type 1, bits[10:7]=0001)
```

```
    //          ID attendu = 0x080 | rxAddress
    BusTer_CanSetMask(BUSTER_Rxm1, BUSTER_Can2_0A, 0x7FF);
    BusTer_CanSetFilter(BUSTER_Rxf2, BUSTER_Can2_0A, 0x080 | rxAddress);
    BusTer_CanSetFilter(BUSTER_Rxf3, BUSTER_Can2_0A, 0x080 | rxAddress);
    BusTer_CanSetFilter(BUSTER_Rxf4, BUSTER_Can2_0A, 0x080 | rxAddress);
    BusTer_CanSetFilter(BUSTER_Rxf5, BUSTER_Can2_0A, 0x080 | rxAddress);

    BusTer_CanActivateNormalMode();

    // LED verte : initialisation OK
    digitalWrite(BUSTER_PIN_STATUS_LED_GREEN, HIGH);
    digitalWrite(BUSTER_PIN_STATUS_LED_RED, LOW);
}

void loop() {
    //-----------------------------------------------------------------
    // 1. Émission digitale
    //    Lire les 8 entrées numériques, les compacter en 1 octet
    //-----------------------------------------------------------------
    INT8U digitalData = 0;
    digitalData |= (INT8U)(digitalRead(BUSTER_PIN_IN_0) << 0);
    digitalData |= (INT8U)(digitalRead(BUSTER_PIN_IN_1) << 1);
    digitalData |= (INT8U)(digitalRead(BUSTER_PIN_IN_2) << 2);
    digitalData |= (INT8U)(digitalRead(BUSTER_PIN_IN_3) << 3);
    digitalData |= (INT8U)(digitalRead(BUSTER_PIN_IN_4) << 4);
    digitalData |= (INT8U)(digitalRead(BUSTER_PIN_IN_5) << 5);
    digitalData |= (INT8U)(digitalRead(BUSTER_PIN_IN_6) << 6);
    digitalData |= (INT8U)(digitalRead(BUSTER_PIN_IN_7) << 7);

    // Envoyer uniquement si la valeur a changé
    if ((int)digitalData != prevDigital && BusTer_CanSendReady(BUSTER_Txb0)) {
        BusTer_CanSend(BUSTER_Txb0, BUSTER_Can2_0A, txAddress, 1, &digitalData);
        prevDigital = (int)digitalData;
    }

    //-----------------------------------------------------------------
    // 2. Émission analogique (uniquement si DIP8=1)
    //    Envoyer seulement si au moins une valeur a changé
    //-----------------------------------------------------------------
    if (analogMode) {
        int ain[4];
        ain[0] = BusTer_AnalogRead(BUSTER_ANALOGINPUT_0);
        ain[1] = BusTer_AnalogRead(BUSTER_ANALOGINPUT_1);
        ain[2] = BusTer_AnalogRead(BUSTER_ANALOGINPUT_2);
        ain[3] = BusTer_AnalogRead(BUSTER_ANALOGINPUT_3);

```

```
        bool changed = (ain[0] != prevAnalog[0]) || (ain[1] != prevAnalog[1]) ||
                       (ain[2] != prevAnalog[2]) || (ain[3] != prevAnalog[3]);

        if (changed && BusTer_CanSendReady(BUSTER_Txb1)) {
            // Encoder 4 valeurs uint16 en little-endian (8 octets au total)
            INT8U analogBuf[8];
            for (int i = 0; i < 4; i++) {
                uint16_t val = (uint16_t)ain[i];
                analogBuf[i * 2]     = (INT8U)(val & 0xFF);
                analogBuf[i * 2 + 1] = (INT8U)((val >> 8) & 0xFF);
            }
            BusTer_CanSend(BUSTER_Txb1, BUSTER_Can2_0A, 0x080 | txAddress, 8, analogBuf);

            prevAnalog[0] = ain[0];
            prevAnalog[1] = ain[1];
            prevAnalog[2] = ain[2];
            prevAnalog[3] = ain[3];
        }
    }

    // ----------------------------------------------------------------
    // 3. Réception digitale (RXB0)
    //    Écrire les 8 sorties numériques depuis l'octet reçu
    // ----------------------------------------------------------------
    if (BusTer_CanMessageReceived(BUSTER_Rxb0)) {
        INT8U  rxBuf[8];
        INT8U  len;
        if (BusTer_CanRead(BUSTER_Rxb0, nullptr , &len, rxBuf) && len >= 1) {
            INT8U d = rxBuf[0];
            digitalWrite(BUSTER_PIN_OUT_0, (d >> 0) & 1);
            digitalWrite(BUSTER_PIN_OUT_1, (d >> 1) & 1);
            digitalWrite(BUSTER_PIN_OUT_2, (d >> 2) & 1);
            digitalWrite(BUSTER_PIN_OUT_3, (d >> 3) & 1);
            digitalWrite(BUSTER_PIN_OUT_4, (d >> 4) & 1);
            digitalWrite(BUSTER_PIN_OUT_5, (d >> 5) & 1);
            digitalWrite(BUSTER_PIN_OUT_6, (d >> 6) & 1);
            digitalWrite(BUSTER_PIN_OUT_7, (d >> 7) & 1);
        }
    }

    // ----------------------------------------------------------------
    // 4. Réception analogique (RXB1, uniquement si DIP8=1)
    //    Écrire les 4 sorties analogiques depuis les 8 octets reçus
    // ----------------------------------------------------------------
    if (analogMode && BusTer_CanMessageReceived(BUSTER_Rxb1)) {
        INT8U  len;
```

```
        INT8U  rxBuf[8];
        if (BusTer_CanRead(BUSTER_Rxb1, nullptr , &len, rxBuf) && len >= 8) {
            int v0 = (int)((uint16_t)rxBuf[0] | ((uint16_t)rxBuf[1] << 8));
            int v1 = (int)((uint16_t)rxBuf[2] | ((uint16_t)rxBuf[3] << 8));
            int v2 = (int)((uint16_t)rxBuf[4] | ((uint16_t)rxBuf[5] << 8));
            int v3 = (int)((uint16_t)rxBuf[6] | ((uint16_t)rxBuf[7] << 8));
            BusTer_AnalogWrite(v0, v1, v2, v3);
        }
    }

    //-----------------------------------------------------------------
    // 5. Gestion des erreurs CAN
    //-----------------------------------------------------------------
    if (BusTer_CanHasError()) {
        digitalWrite(BUSTER_PIN_STATUS_LED_RED, HIGH);
        digitalWrite(BUSTER_PIN_STATUS_LED_GREEN, LOW);
        BusTer_CanClearError();
    } else {
        digitalWrite(BUSTER_PIN_STATUS_LED_RED, LOW);
        digitalWrite(BUSTER_PIN_STATUS_LED_GREEN, HIGH);
    }
}
```

Listing 1 – `labo4.ino` – programme principal du laboratoire